V-CoT: Vision Chain-of-Thought for Structured Reasoning Tasks

Anonymous ACL submission

Abstract

Multi-modality promises to unlock further uses for large language models. Recently, the state-of-the-art language model GPT-4 was enhanced with vision capabilities. We carry out an evaluation of GPT-4V and five other baselines on structured reasoning tasks, such as mathematical reasoning, visual data analysis, and code generation. We show that visual Chain-of-Thought, an extension of Chain-of-Thought to multi-modal LLMs, yields significant improvements over the vanilla models in both prompting and instruction tuned settings. We also present a categorized analysis of scenarios where these models perform well and where they struggle, highlighting challenges associated with coherent multimodal reasoning.

1 Introduction

Visual instruction tuning (Liu et al., 2023) allows language models to answer questions about images. By leveraging a dataset of captioned images, strong language models (like GPT-4) can be used to generate high-quality training data that spans conversation, detailed descriptions and reasoning about concepts found in these images. The visual instruction tuned model is then able to reason (to some extent) about the input image (Zhang et al., 2023c). OpenAI GPT-4V is the state-of-the-art language model GPT-4 (OpenAI, 2023) augmented with a vision component.

Example 1. The following image and part of the associated complex reasoning instruction are part of the LLaVA dataset (Liu et al., 2023). The image and **Q** are provided to the model, and it is trained to return **A**.



Q: What can be inferred about the relationship between these zebras?
A: The zebras are standing near each other and nuzzling their heads together in their enclosure. This behavior suggests a bonding moment or a sign of affection.

In large language models, such as GPT-4, explicitly eliciting reasoning significantly improves performance. A popular method is Chain-of-Thought (CoT) where a literal instruction to "think step by step" causes the model to first generate some thoughts, and then use these thoughts to generate a final answer (Wei et al., 2022). Similar to complex reasoning in LLaVA, multi-modal Chain-of-Thought (m-CoT) (Zhang et al., 2023c) leverages generating CoT data to fine-tune a multi-modal model to elicit reasoning about and answer scientific questions over graphics.

In this work, we propose V-CoT, a vision chain-of-thought technique that first extracts task specific propositions from the image followed by reasoning about the task using the propositions. We explore the extent to which existing can *reason* in different domains, such as code and math, when part of the inputs are images or can be represented as images. More specifically, we evaluate these models in four domains.

1. Mathematical questions with visual context from the MathVista dataset (Lu et al., 2023).

2. Questions about data charts from the ChartQA dataset (Masry et al., 2022).

3. Abstraction and reasoning problems over objects in grids from the ARC dataset (Chollet, 2019, 2023).

4. Generating SQL from NL given a rendered table from the Spider dataset (Yu et al., 2018).

Each of these domains requires specific reasoning that might not be present in the training data.

Rather than fine-tuning a dedicated model, like m-CoT, we adapt the classical CoT approach to multi-modal language models, and show that it significantly outperforms the base (visual instruction tuned) model for both prompting and instruction

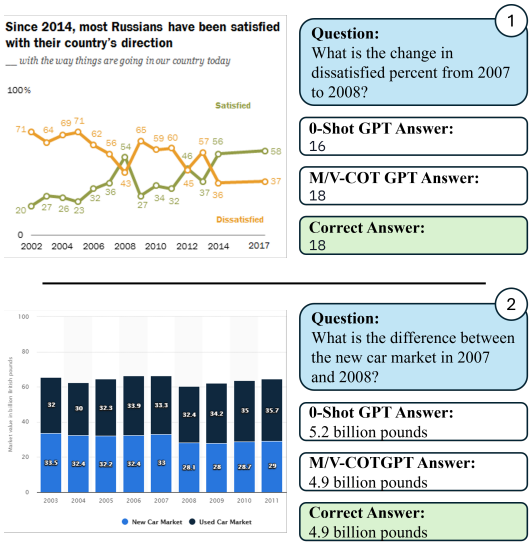


Figure 1: Without reasoning, the model does not find the answer. Both m-CoT and v-CoT elicit sufficient reasoning.

tuning approaches. Besides using the exact reasoning structure from m-CoT, we introduce an improved, three-step reasoning process to further improve performance, called *visual Chain-of-Thought* (v-CoT). In v-CoT, we instruct the model to (1) extract relevant information about the image in the form of propositions, (2) use these propositions to reason about the problem, and (3) state the final answer. Note that step (1) is not the same as a regular image captioning step, as it is also conditioned on the task along with the image.

Example 2. Figure 1 shows two examples where without reasoning, the model is not able find the correct answer.

Example 3. Figure 2 shows an example of reasoning with m-CoT and v-CoT. By first describing relevant information, the model is then able to reason about that information to obtain the right result.

Our results show that for MathVista, ChartQA and Spider, GPT-4V outperforms baselines, including CoT and Program-of-Thought (PoT) prompting with GPT-4 over captions generated by InstructBlip (Dai et al., 2023). For ARC, we find that directly prompting over the vision component actually results in lower performance than captioning and CoT or PoT. We find that using v-CoT improves 1.5 – 9.3 percentage points over vanilla GPT-4V, and 1.1 – 4.7 percentage points compared with the m-CoT prompt over both prompting and instruction tuned variants.

This work makes the following contributions:

- A comparative evaluation of multimodal models and (captioning + GPT-4) on structured reasoning tasks.
- Visual Chain-of-Thought (v-CoT) as an extension of CoT to multi-modal LLMs that first asks the model to extract relevant properties to reason over. v-CoT improves on vanillam-odels by 1.5 – 9.3 percentage points.
- A thorough analysis of recurring patterns to understand performance of these models.

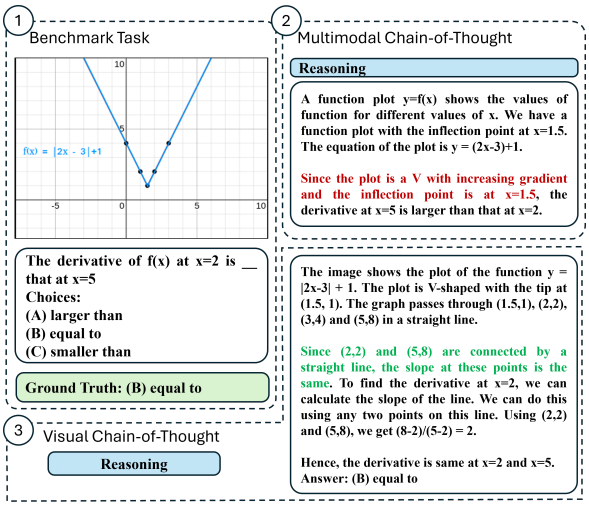


Figure 2: (1) An example task, (2) the output generated with m-CoT, and (3) with v-CoT. Red and green text highlight incorrect and correct reasoning, respectively.

2 Related Work

Prior work has explored combining multiple modalities using transformer-based models, including visual and text (Hu et al., 2023; Dai et al., 2023; Li et al., 2023; Lin et al., 2023), speech and text (Zhang et al., 2023a), video and audio (Zhang et al., 2023b; Maaz et al., 2023), and the union of these modalities (Wu et al., 2023). In addition, there is a long history of models taking inputs in one modality (like image or text) and generating outputs in another (like text or images, respectively) (Hossain et al., 2019; Reed et al., 2016). For the image-text multimodal models, visual question-answering (VQA) (Antol et al., 2015; Agrawal et al., 2016) and conditioned captioning (Li et al., 2023; Dai et al., 2023) are the most popular tasks where the input is an image and optionally some text and the output is text conditioned on botht he input image and text. In this work, we present an empirical evaluation of an existing state-of-the-art vision-text

multimodal model (GPT-4V) on structured tasks like mathematical reasoning and code generation, and consider a setting similar to VQA where both images and text are used as input and output is text.

To evaluate these multi-modal models a large variety of benchmarks have been introduced, including VQA (Antol et al., 2015) and RefCOCO (Yu et al., 2016). In domains of structured reasoning, recent datasets include MathVista (Lu et al., 2023), which collects a large number of mathematical reasoning tasks, and ChartQA (Masry et al., 2022), which presents questions to be answered based on data analysis plots. Recently, MMMU (Yue et al., 2023) introduced college-level multi-modal tasks across a large range of domains including technical areas. Our evaluation also considers rendering existing structured inputs (tables) as images to perform text-to-SQL generation over them, using the popular Spider dataset (Yu et al., 2018).

Chain-of-Thought (Wei et al., 2022) reasoning traces were used to fine-tune a multi-modal CoT model Zhang et al. (2023c). We take inspiration from m-CoT in our prompting experiments with GPT-4V. Leveraging a larger model (GPT-4), our visual guided prompting techniques are aimed at generalizing better to new modes of reasoning.

3 Visual Chain of Thought (V-CoT)

In this section we define visual chain of thought and our implementation. Specifically, we first formally describe structured reasoning. Then we introduce our approach to generate appropriate training data, which we use to pre-train on such structured reasoning inputs. Finally, we discuss fine-tuning to learn this end-to-end.

3.1 Preliminaries

Definition 3.1 (Multimodal task)**.** We define a multimodal structured reasoning task $\mathcal{T}$ as a triplet of (`text input` $\mathcal{T}_{\text{text}}$, `vision input` $\mathcal{T}_{\text{vision}}$, `text answer` $\mathcal{A}$). The goal of a multimodal model $\mathcal{M}$ is to take $(\mathcal{T}_{\text{text}}, \mathcal{T}_{\text{vision}})$ and produce $\mathcal{A}$. For example, figure 2 shows an example multimodal task.

Definition 3.2 (Proposition)**.** Let a proposition $\mathcal{P}$ be a triplet $\{e, p, v\}$ where e is a visual entity in an image, p is a property of that entity, and v is the value of that property. We say an image $\mathcal{T}_{\text{vision}}$ satisfies a proposition, if associated triplet is true.

Example 4. In Figure 2, `{line, equation, |2x-3|+1}` is a valid proposition. Similarly, `{line, inflection, (1.5, 1)}` and `{line, passes, (1.5, 1)}` are also valid propositions. While `{line, passes, (2, 1)}` is an invalid proposition, because the line does not pass through (2, 1).

Definition 3.3 (Task-oriented propositions)**.** Given a multimodal task $\mathcal{T}$, let $\mathcal{P}$ be the set of propositions that are satisfied by the associated image $\mathcal{T}_{\text{vision}}$. Let the subset of $\mathcal{P}$ that are relevant to the task text and answer be $\mathcal{P}_{\mathcal{T}} \subset \mathcal{P}$. We call these task-oriented propositions. We refer to proposition refinement as the act of filtering down $\mathcal{P}$ to $\mathcal{P}_{\mathcal{T}}$.

Example 5. Consider the task shown in Figure 3, for the task $\mathcal{T}$ of finding the derivative at x=2 and x=5, one proposition $p \in \mathcal{P}_{\mathcal{T}}$ is `{line, equation, |2x-3|+1}`, as this can be used to define the line.

Below, for completeness, we summarize the definition of reasoning path in the context of chain-of-thought reasoning (Wei et al., 2022).

Definition 3.4 (Reasoning path)**.** We define a reasoning path $\mathcal{R}$ over a task $\mathcal{T}$ as a sequence of natural language-based deductions, which start from a set of axioms and concludes with an answer.

Now that we have defined key high-level components, we proceed to define v-COT

Definition 3.5 (V-Cot)**.** Given a multimodal task $\mathcal{T} = (\mathcal{T}_{\text{text}}, \mathcal{T}_{\text{vision}}, \mathcal{A})$, v-CoT (1) extracts a set of task-oriented propositions, $\mathcal{P}_{\mathcal{T}}$, (2) constructs a reasoning path starting from $\mathcal{T}_{\text{text}}$ and $\mathcal{P}_{\mathcal{T}}$ as axioms, (3) produces an output $\hat{\mathcal{A}}$. We say v-CoT correctly answered the task if $\hat{\mathcal{A}} = \mathcal{A}$.

In contrast to m-CoT, v-CoT does not allow the model to initially generate free-form natural language rationale for the task. Instead v-CoT constrains the first set of axioms used in the reasoning path to be structured propositions, along with the task text instructions. In practice, v-CoT does not need to extract $\mathcal{P}_{\mathcal{T}}$ perfectly – different propositions may be of varying importance and a superset of propositions may also be helpful (as long as it does not distract the model in the reasoning path). Our results (Table-3) show that despite not enforcing completeness or precision of proposition extraction, v-CoT results in improvements.

3.2 Generating `v-CoT` Training Data

We now describe our approach to generate v-CoT training data, in the form of 5-tuples of (`text input` ($\mathcal{T}_{text}$), `vision input` ($\mathcal{T}_{vision}$), `task-oriented propositions` ($\mathcal{P}_{\mathcal{T}}$), `v-CoT reasoning` ($\mathcal{R}$), `output` ($\mathcal{A}$)). The input to

our data generation process is a dataset of standard multimodal tasks $\mathcal{T}$ (see Definition 3.1).

3.2.1 Proposition ($\mathcal{P}$) Generation

We first generate structured propositions $\mathcal{P}$ from the image input $\mathcal{T}_{vision}$. We introduce two approaches to extracting these propositions:

1. **Program-analysis based proposition extraction**. Since we focus on structured task-domains, where inputs themselves are structured (e.g. tables, plots), many of our image inputs already have a structured definition available that is amenable to analysis. For example, if plot showing the population increase of different continents was generated programmatically, we can extract `(Africa, population increase, 13%)` using simple syntactic analysis.

2. **Image-analysis based proposition extraction**. We can also mine propositions directly from the image $\mathcal{T}_{vision}$. To do so, we use a small captioning model (InstructBLIP) to generate propositions, prompted with $\mathcal{T}_{vision}$, $\mathcal{T}_{text}$, and $\mathcal{A}$. To mitigate the occurence of free-form text propositions (`"the force applied is 5N from left"`) or incorrectly structured propositions (`"(force, 5N from left)"`), we perform a second pass using a language model. This pass prompts the model with the $\mathcal{T}_{text}$ and $\mathcal{A}$ and the candidate propositions and can reformat candidate propositions or add additional information.

We employ both methods, if information is available, to complement the propositions. Otherwise, we only apply the approach possible (e.g. if we only have image but not its structured definition).

3.2.2 Reasoning Path ($\mathcal{R}$) Generation

To generate the `v-CoT` reasoning path ($\mathcal{R}$), we prompt a language model (gpt4-turbo) with the task description $\mathcal{T}_{text}$, structured propositions $\mathcal{P}$, and target answer $\mathcal{A}$. We ask the model to generate the reasoning $\mathcal{R}$ for the target answer using $\mathcal{P}$ and $\mathcal{T}_{text}$. We also instruct the model to tag the propositions directly in its reasoning and add any propositions that are useful but not in the candidate set originally included. Note that the image is no longer used in this step, so it need not be carried out by a multimodal model.

3.2.3 Proposition Refinement ($\mathcal{P}_T$)

Note that the structured propositions generated may not be relevant to the task. To refine this set to task-oriented propositions we filter down to propositions directly referenced in the reasoning path $\mathcal{R}$. At the end of this procedure, we have a set of 5-tuples necessary to carry out v-COT training.

3.3 V-COT Instruction Tuning

To train a model to perform V-COT, we break up training into pretraining and fine-tuning tasks. We use the dataset of 5-tuples generated in Section 3.2.

3.3.1 Pretraining

For pretraining, we use two separate pretraining objectives, adapted from the instruction-tuning approach introduced by Zhang et al. (2023c).

1. Objective 1: inputs are of the form (`task description` $\mathcal{T}_{\texttt{text}}$, `image` $\mathcal{T}_{\texttt{vision}}$) and the output are the task-oriented propositions $\mathcal{P}_{\mathcal{T}}$. The aim of this objective is to teach the model to extract valid structured propositions, conditioned on the task image and text.

2. Objective 2: inputs are of the form (`task description` $\mathcal{T}_{\texttt{text}}$, `task-oriented propositions` $\mathcal{P}_{\mathcal{T}}$, `image` $\mathcal{T}_{\texttt{vision}}$ and the output is the reasoning path $\mathcal{R}$. The aim of this objective is to teach the model to combine text and propositions into reasoning paths.

During pretraining, we randomly sample one of these two objectives per-batch.

3.3.2 Finetuning

In the finetuning phase, the model input is the multimodal task (excluding the task answer) $\mathcal{T} \setminus \mathcal{A}$ and the output is the concatenation of the reasoning path $\mathcal{R}$ and the target answer $\mathcal{A}$.

3.4 Prompting

V-CoT can also be employed in prompting-only approach. In this setting, we prompt the model to first generate the `v-CoT` task-oriented propositions $\mathcal{P}_{\mathcal{T}}$ and then generate a reasoning path $\mathcal{R}$ using the propositions before generating the final answer $\mathcal{A}$. The prompt structure for `v-CoT` is shown in Figure **??**, and a sample prompt for a chart question answering task is shown in Figure 3.

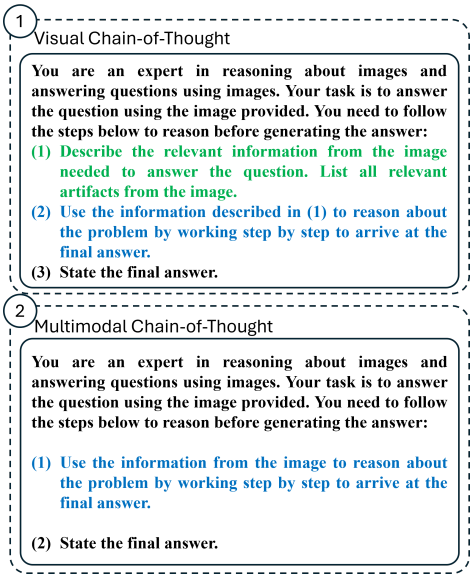


Figure 3: Prompt structure for v-CoT (1) and m-CoT (2) prompts. Blue highlights the shared m-CoT instruction and green highlights our extension.

4 Evaluation Setup

We now describe our evaluation setup. First, we focus on explaining our baselines, followed by the problems on which these approaches were evaluated. Finally, we provide more details on the v-COT training data generation for each benchmark.

4.1 Methods

4.1.1 m-CoT and v-CoT

Figure 3 compares the corresponding prompts for v-CoT and m-CoT. The latter works as traditional CoT but over both text and image inputs. Specifically, the instruction to perform reasoning specifies that the image can be used to arrive at final answer in a step-by-step fashion. Both approaches can be used in a prompting-only setup or with further fine-tuned on the same training datasets.

4.1.2 Instruction tuning

Sphinx (Lin et al., 2023) is a large multimodal model (LMM) with multi-purpose visual instruction-following capabilities Sphinx has been trained on a variety of vision and language alignment tasks like object detection, visual question answering and region level captioning and achieves state-of-the-art performance on these.

Blip2 (Li et al., 2023) is a text and image pre-training technique that bootstraps vision-language pre-training from off-the-shelf frozen pre-trained image encoders and frozen large language models.

Dataset	Tasks	Image	Task
ChartQA (test)	302	Chart	VQA
MathVista (testmini)	200	Diagram	VQA
ARC (test)	80	Blocks	Pattern
Spider (test)	206	Table	Code generation

Table 1: Tasks from our evaluation benchmarks, we present the total number of tasks, image description, and task description.

The model combines existing vision and language models and pre-trains them on language and vision alignment tasks. Blip-2 has been trained on image-language tasks like conditional image captioning.

InstructBlip (Dai et al., 2023) is an extension that applies instruction tuning to Blip-2 for language and vision tasks (visual question answering).

4.2 Benchmarks

This section describes the benchmark tasks used in our evaluation. Table 1 summarizes the benchmarks, and Figure 4 presents example tasks.

4.2.1 Mathematical Reasoning

We evaluate mathematical reasoning given a visual context with the recently introduced MathVista dataset (Lu et al., 2023). These problems require reasoning over images which may contain diagrams, tables or other images. Each task consists of an image and question pair. Questions in MathVista can be multiple-choice, free-form with single value as result, or free-form with a list as a result. We sampled tasks from the testmini split of MathVista, which resulted in multiple choice and single-value free-form questions[1].

4.2.2 Visual Data Analysis

We evaluate the ability to answer questions based on charted data—an important skill for visual data analysis—using the ChartQA dataset (Masry et al., 2022). Each tasks consists of a plot as a rendered image and a question about this plot. These questions are short and objective, with 2-3 word or numeric value answers. An example question is "*How many crimes were committed in 2020?*" The question is passed in the prompt, preceded by the following instruction:

```
Answer the question using the image.
Only give the exact answer in 2-3 words
or as a numeric value.
```

[1]testmini has only 2 free-form list questions

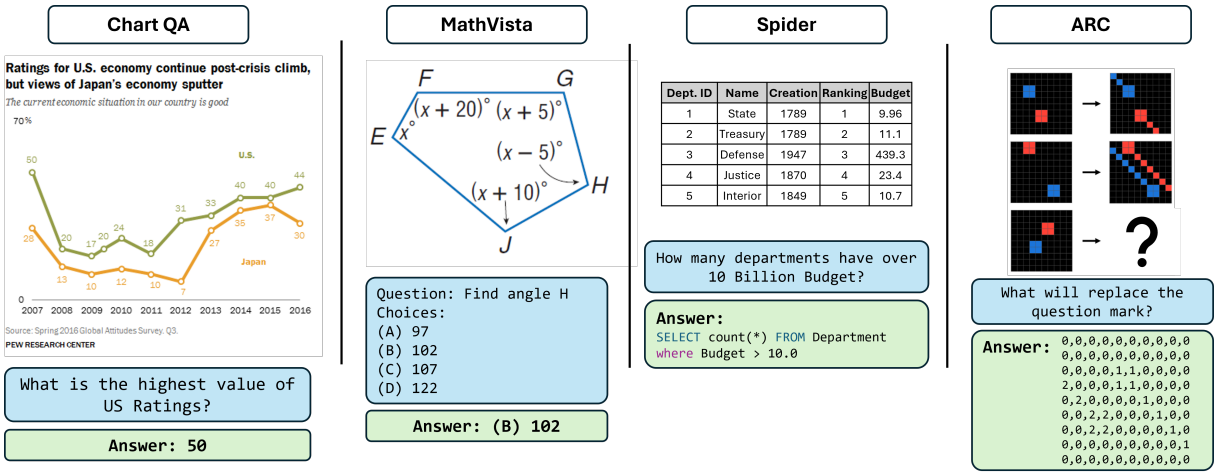


Figure 4: Sample tasks from each benchmark dataset. We show the image and the associated text prompt for the dataset along with the correct answer for the task. For Chart QA and MathVista the answer is a choice or numeric value; For Spider the answer is the correct SQL query; For ARC the output is the correct pixel matrix ($0 \rightarrow black; 1 \rightarrow red; 2 \rightarrow blue$).

4.2.3 Visual Abstraction and Extrapolation

Each ARC (Chollet, 2023) task consists of a set of examples, where an example consists of an input and output grid, and a new input—the model should predict the associated output grid. Grids are represented as comma-separated-value (CSV) table of integers. Sample instruction:

```
Generate the transformed representation that will replace the question mark by looking at the example figure transformation. Generate as a <gridsize> Grid where 0 denotes black, 1 denotes red and 2 denotes blue.
```

4.2.4 Code Generation

We evaluate NL-to-SQL generation in a multi-modal setting using Spider dataset (Yu et al., 2018). Each task input consists of a relational database and natural language question pair, and the task output is the SQL code needed to answer that question. To turn Spider tasks into multi-modal tasks, we consider only tasks over a single table and we render the first 50 rows into an image using `matplotlib.pyplot.table` (Hunter, 2007). We use the following instruction:

```
Generate SQL query for the given user question. The relevant table is shown in the image and the metadata is included.
Metadata: <Metadata>
Question: <Question>
```

4.3 Training Datasets

We generate `v-CoT` instruction tuning data for 3 multimodal problems, (1) question answering (QA) over charts; (2) visual mathematical and analytical problems; (3) text guided SQL program generation over single tables. All three dataset originally contain multimodal tasks in the form of `{text input, vision input, target}` triplet. Table 2 summarizes the statistics of the generated training data.

4.3.1 Code Generation

We use (Zhong et al., 2017) as the original source dataset. The source dataset contains `(text instruction, table, SQL query)` triplets. The tables are originally in `HTML` encoding. To generate images, we use `matplotlib` to convert the tables to image. We generate program analysis based propositions as described in Section 3.2.1.

Example 6. Consider the SQL query `SELECT "Name" from "Students" where "ID" == "X12"`, our proposition mining will return (Table Students, Has Name, "Name"), (Name, Has Value, X12), (Table Students, Has Name, ID).

In addition, we use the model-prompting approach described in Section 3.2.1 to augment these.

4.3.2 MathVista

We use math word problem dataset (Lu et al., 2023) (test-split) as the source dataset. The source dataset has triplets of `(question text, figure image, answer)`. In this case, as we do not have the underlying source code for the image, we use the image-analysis approach described in Section 3.2.1.

Problem	Samples	$Len(\mathcal{T})$	$Len(\mathcal{P}')$	$Len(\mathcal{R})$
Text to SQL	67.5K	130.5	19.5	534.8
MathVista	15.6K	167.5	34.3	872.1
ChartQA	23.1K	193.4	23.6	645.2

Table 2: Table summarizing the statistics of generated training data for v-CoT reasoning based instruction tuning. Text length denotes the average character counts of the input task text. $Len(\mathcal{P}')$ and $Len(\mathcal{R})$ denote the average of v-CoT propositions and character count of the v-CoT reasoning respectively, in training data.

4.3.3 ChartQA

We use (Masry et al., 2022) as the source dataset for this domain. The source dataset has triplets of `(question text, chart generation code, answer)`. Charts in this dataset are defined in VizQL (Hanrahan, 2006), a domain-specific language for charts. We render these charts as images using `matplotlib`. Similarly to SQL, we extract propositions from the VizQL definitions. As before, we also merge propositions with those from the model-based approach described in Section 3.2.1.

5 Results

We aim to answer the following research questions through experimentation. We also do a case study (**CS1**) on qualitative analysis of v-CoT reasoning.

RQ1 Does v-COT improve multimodal model performance on structured reasoning tasks?

RQ2 Does v-CoT improve reasoning quality and correctness?

RQ3 What is the impact of various components in v-CoT?

5.1 RQ1: Performance of v-CoT

To measure the improvements with structured reasoning from v-CoT we report the task performance of different large multimodal models on 4 different multimodal (text-image) datasets.

To highlight improvements, we report three variants for all models - (1) **CoT**: with normal chain of thought; (2) **m-CoT**: with multimodal chain of thought; and (3) **v-CoT**: with visual chain of thought (proposed). To ensure the completeness of the analysis, we consider both inference only and fine tuned versions of the models. For inference only setting, the model is prompted to generate the reasoning paths before generating the output.

Table 3 shows the result of v-CoT and other prompting techniques on all benchmark datasets. v-CoT performs better across both inference only (+3.2%) and fine tuned (+5.4%) setting.

5.2 RQ2: Reasoning Analysis

We see that v-CoT performs better that other reasoning strategies in task performance. One key aspect of reasoning based task solving is the quality of the reasoning generated by the models and how it affects performance. We analyze the quality of reasoning produced with different reasoning strategies for GPT-4V (best inference model) and Llava (best instruction tuned model).

We sample 100 tasks from each dataset and extract the (reasoning $\mathcal{R}$, $\mathcal{A}$) pairs from these. We annotate the correctness of $\mathcal{R}$ and $\mathcal{A}$. Figure 5 shows the result of this analysis. The correctness of both the answer and reasoning increases for v-CoT over m-CoT. This difference is largest for ARC (+15.0%) and smallest for Text to SQL (+2.3%).

To understand the completeness of the reasoning generated, we study the impact of replacing the image with the reasoning $\mathcal{R}$ generated by GPT-4V. For this, we append $\mathcal{R}$ to the task description $\mathcal{T}$ and feed the complete text prompt to GPT-4. We do this analysis for both v-CoT and m-CoT. We find that v-CoT reasoning is more complete and leads to a higher performance when the image is omitted compared to m-CoT.

5.3 RQ3: Ablations

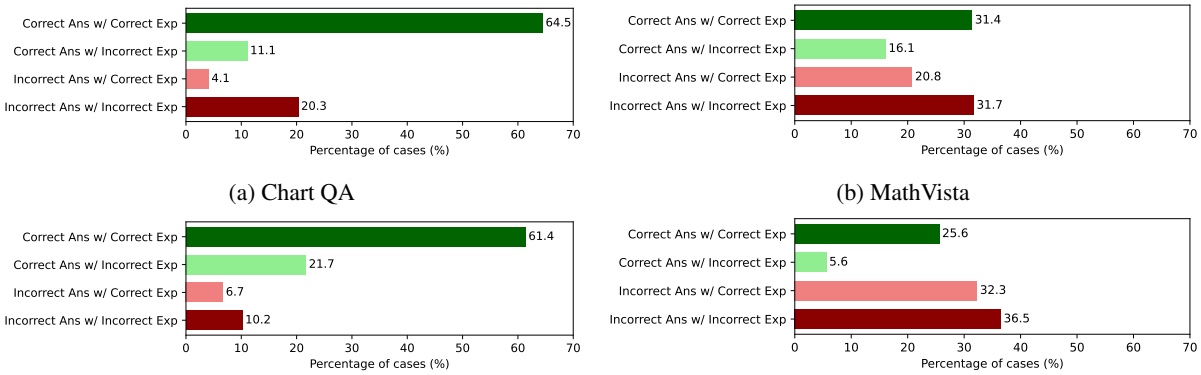
We perform ablations to understand the impact of each component in v-CoT. Table **??** show results for removing pre-training objectives. We find that the proposition extraction pre-training objective has the highest impact on performance (+7.5%) while the proposition sourced from program analysis are the most impactful (+6.3%).

6 CS1: Qualitative Analysis

We manually annotated GPT-4V results for whether the reasoning provided was correct or not and whether the answer was correct or not. Figure 5 presents our findings. Across all datasets, we observe that there are tasks that GPT-4V correctly answers, despite having incorrect reasoning. Similarly, we find that despite producing correct reasoning, the model can still produce an incorrect answer. This issue is particularly frequent in both the MathVista and ARC tasks.

System		ChartQA			MathVista			Spider			ARC		
Model Name	**Technique**	CoT	mCoT	vCoT	CoT	mCoT	vCoT	CoT	mCoT	vCoT	CoT	mCoT	vCoT
Inference based models													
BLIP2	Inference	31.4	33.5	36.9	23.1	29.1	30.5	78.5	80.1	82.2	22.6	44.3	48.2
InstructBLIP	Inference	54.5	55.3	56.4	24.7	30.4	30.8	80.1	82.7	83.5	23.4	47.3	50.5
Sphinx	Inference	63.4	65.1	68.3	33.2	35.4	38.8	84.1	85.5	86.7	27.2	28.5	30.2
Llava	Inference	66.4	68.3	70.5	40.4	44.3	46.1	81.8	82.3	85.1	28.5	29.1	32.5
GPT-4V	Inference	72.3	75.6	79.2	43.5	47.5	49.1	82.1	84.3	86.2	29.8	31.2	51.2
Instruction tuned models													
BLIP2	Finetuned	33.9	36.0	39.4	25.6	31.6	33.0	81.0	82.6	84.7	25.1	46.8	50.7
InstructBLIP	Finetuned	57.0	57.8	58.9	27.2	32.9	33.3	82.6	85.2	86.0	25.9	49.8	53.0
Sphinx	Finetuned	65.5	66.7	70.1	35.4	36.7	38.9	85.2	86.5	87.7	28.0	28.7	30.9
Llava	Finetuned	76.5	78.4	80.5	45.1	46.8	48.3	83.5	85.7	87.3	29.1	30.5	48.4

Table 3: Comparison of v-CoT, m-CoT and CoT for different multimodal models in inference and fine tuned setting. We report task success rate for different datasets. v-CoT outperforms other reasoning techniques for both inference and fine tuned models.



(a) Chart QA

(b) MathVista

(c) Spider

(d) ARC

Figure 5: Manual analysis of GPT-4V + VCoT on our sampled tasks. We manually annotate the reasoning and the final answer separately for all benchmark dataset and present the analysis. For Chart QA and Spider, we find that in 72 – 85 % cases the model generates the right explanation and also the correct answer. For MathVista and ARC that require more complex reasoning, we find that the model struggles with only 51 – 58 % reasoning being correct.

	Chart	**Math**	**Spider**	**ARC**
$\mathcal{M}$ - Proposition Obj	72.8	41.2	80.1	38.7
$\mathcal{M}$ - Reasoning Obj	72.5	40.6	79	40.1
$\mathcal{M}$ - Program Props	74.3	42.1	80.6	43.4
$\mathcal{M}$ - Model Props	78.4	45.6	85.3	47.6
$\mathcal{M}$	**80.5**	**48.3**	**87.3**	**48.4**

Table 4: Performance of v-CoT ($\mathcal{M}$) on finetuned Llava with different ablations. Rows 1 and 2 ablate the pre-training objectives while rows 3 and 4 ablate the proposition sources. Full $\mathcal{M}$ outperforms all ablations.

We find that for both MathVista and ChartQA, GPT-4V outperforms other approaches. Interestingly, for the ARC dataset, we observe that using the multi-modal version of GPT-4 actually produces worse results, compared to generating textual descriptions and then applying GPT-4 plus CoT or PoT (the latter of which improves results most). Overall, the ARC dataset represents a challenging task. On the Spider dataset, we find that including image improves performance by 0.8 percentage points, while adding v-CoT improves by 1.9.

7 Conclusion

We show that our visual Chain-of-Thought (v-CoT) prompt improves performance by first instructing the model to analyse the image, conditioned on the task at hand, and then use this analysis to reason towards a final result. Our experiments compare GPT-4V (with and without v-CoT) to multiple existing multi-modal models. We find that v-CoT outperforms other baselines in three datasets, but that it still struggles with the ARC dataset, which requires abstraction and extrapolation.

8 Limitations

First, we consider a subset of structured reasoning tasks, performance on other tasks may vary. Similarly, we sample 20% of tasks per dataset to mitigate computational costs—while this sampling was done uniformly at random, it is possible that performance would change for other tasks. While manually annotating results, we also identified that there are tasks in the various datasets (particularly ChartQA and MathVista) that are ambiguous or may have incorrect ground truth labels. We take all labeling as given. All our tasks are framed in English (images with English text, tasks with English text). Evaluating GPT-4V and other models on multilingual tasks remains future work.

References

Aishwarya Agrawal, Jiasen Lu, Stanislaw Antol, Margaret Mitchell, C. Lawrence Zitnick, Dhruv Batra, and Devi Parikh. 2016. Vqa: Visual question answering. *Preprint*, arXiv:1505.00468.

Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. 2015. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

F. Chollet. 2023. The abstraction and reasoning corpus (arc). Last-accessed: 15-11-2023.

François Chollet. 2019. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*.

Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. 2023. Instructblip: Towards general-purpose vision-language models with instruction tuning. *Preprint*, arXiv:2305.06500.

Pat Hanrahan. 2006. Vizql: a language for query, analysis and visualization. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pages 721–721.

MD Zakir Hossain, Ferdous Sohel, Mohd Fairuz Shiratuddin, and Hamid Laga. 2019. A comprehensive survey of deep learning for image captioning. *ACM Computing Surveys (CsUR)*, 51(6):1–36.

Wenbo Hu, Yifan Xu, Y Li, W Li, Z Chen, and Z Tu. 2023. Bliva: A simple multimodal llm for better handling of text-rich visual questions. *arXiv preprint arXiv:2308.09936*.

J. D. Hunter. 2007. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *Preprint*, arXiv:2301.12597.

Ziyi Lin, Chris Liu, Renrui Zhang, Peng Gao, Longtian Qiu, Han Xiao, Han Qiu, Chen Lin, Wenqi Shao, Keqin Chen, et al. 2023. Sphinx: The joint mixing of weights, tasks, and visual embeddings for multi-modal large language models. *arXiv preprint arXiv:2311.07575*.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *NeurIPS*.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2023. Mathvista: Evaluating math reasoning in visual contexts with gpt-4v, bard, and other large multimodal models. *Preprint*, arXiv:2310.02255.

Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. 2023. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*.

Ahmed Masry, Xuan Long Do, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279, Dublin, Ireland. Association for Computational Linguistics.

OpenAI. 2023. Gpt-4 technical report. *ArXiv*, abs/2303.08774.

Scott Reed, Zeynep Akata, Xinchen Yan, Lajanugen Logeswaran, Bernt Schiele, and Honglak Lee. 2016. Generative adversarial text to image synthesis. In *International conference on machine learning*, pages 1060–1069. PMLR.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2023. Next-gpt: Any-to-any multimodal llm. *arXiv preprint arXiv:2309.05519*.

Licheng Yu, Patrick Poirson, Shan Yang, Alexander C Berg, and Tamara L Berg. 2016. Modeling context in referring expressions. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The*

Netherlands, October 11-14, 2016, Proceedings, Part II 14, pages 69–85. Springer.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium. Association for Computational Linguistics.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *arXiv preprint arXiv:2311.16502*.

Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu. 2023a. Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities. *arXiv preprint arXiv:2305.11000*.

Hang Zhang, Xin Li, and Lidong Bing. 2023b. Video-llama: An instruction-tuned audio-visual language model for video understanding. *arXiv preprint arXiv:2306.02858*.

Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. 2023c. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923*.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *Preprint*, arXiv:1709.00103.

A Baselines

A.1 m-CoT and v-CoT

Figure 3 compares the corresponding prompts for v-CoT and m-CoT. The latter works as traditional CoT but over both text and image inputs. Specifically, the instruction to perform reasoning specifies that the image can be used to arrive at final answer in a step-by-step fashion.

Both approaches can be used in a prompting-only setup or with further fine-tuned on the same training datasets.

A.2 Instruction tuning

Sphinx (Lin et al., 2023) is a large multimodal model (LMM) with multi-purpose visual instruction-following capabilities Sphinx has been trained on a variety of vision and language alignment tasks like object detection, visual question answering and region level captioning and achieves state-of-the-art performance on these.

Blip2 (Li et al., 2023) is a text and image pre-training technique that bootstraps vision-language pre-training from off-the-shelf frozen pre-trained image encoders and frozen large language models. The model combines existing vision and language models and pre-trains them on language and vision alignment tasks. Blip2 has been trained on various image-language tasks like conditional image captioning.

InstructBlip (Dai et al., 2023) is an extension that applies instruction tuning to Blip-2 for language and vision tasks like visual question answering.

A.3 Captioning + Prompting

InstructBlip + GPT4 with **Chain-of-thought (CoT)** (Wei et al., 2022) and **Program-of-thought (PoT)** (Chen et al., 2023) prompting of GPT-4. CoT and PoT have shown state-of-the-art performance in text and code generation tasks. To incorporate images, we first generate a caption for the image using InstructBlip, add the caption to the input prompt for GPT-4, and provide instructions for CoT or PoT.

B Discussion of Patterns

We discuss recurring patterns we observed in our manual analysis of results.

B.1 Mathematical Reasoning

We identify the following recurring patterns in the MathVista failures.

- Incorrect computation over derived values. Figure 7b shows an example where the model identifies correct values and constraints for a trigonometric problem, but fails to compute the final answer.
- Inferring numerical labels. When the model has to infer numerical labels, like aggregates in bar charts, the model often picks wrong values and fails to answer, despite having correct reasoning. This is shown in Figure 7c.

B.2 Visual Data Analysis

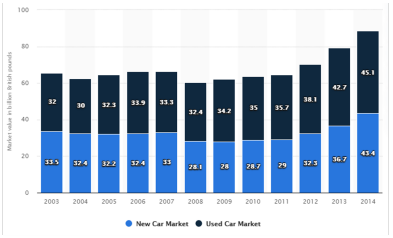
We find that in ChartQA, multiple failures can be attributed to a few emergent behaviors, which are highlighted below. These patterns are also illustrated in Figure 6.

- Incorrect computation over chart-derived values. For example, take the question "*What is the growth from 2010 to 2011?*" over the chart in Figure 6a. Despite identifying each of these values correctly, it fails to compute the growth amount by subtracting the value of 2007 from that of 2008.
- Linking colors/visual elements. For example, when asked a question about a plot with two lines of varying shade of yellow, the model flips their labeling and answers with the wrong value (see Figure 6b).
- Computations that require extracting values from both image and the query. For example, extracting a rate from the text query and using this to identify differences in a plot (see Figure 6c).

B.3 Abstraction and Extrapolation

In the ARC dataset, all methods struggle to generate the correct output since it requires complex reasoning over visual patterns. We find the following to be a common pattern.

- Incorrect grid perception. Despite using v-CoT, we find cases where GPT-4V does not correctly identify block positions. For example, v-CoT may place blocks at the wrong vertical offset (see Figure 8).

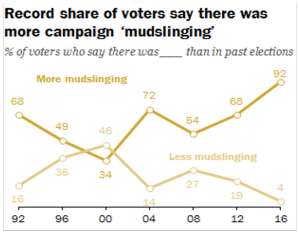


Question:
What is the difference between the new car market in 2007 and 2008?

GPT Answer:
5.2 billion pounds

Correct Answer:
4.9 billion pounds

(a) Computation Mistake

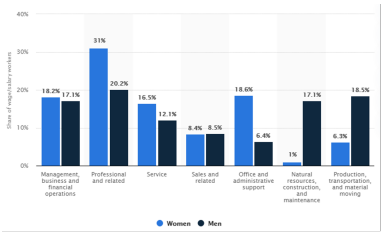


Question:
What percent of voters think there were more mudslinging in 2000?

GPT Answer:
46

Correct Answer:
34

(b) Color Inferring Mistake



Question:
How many sectors have a gender gap of more than 10%?

GPT Answer:
3

Correct Answer:
4

(c) Text and Image Reasoning

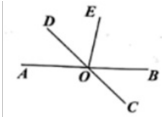
Figure 6: Observed patterns in GPT-4V behavior for ChartQA tasks. (a) shows an example where the model generates the correct reasoning, i.e. to subtract new car market share of 2008 from 2007, but makes a computation error in the final answer. (b) shows an example where the model generates the correct reasoning but applies it to the light yellow line instead of the dark yellow. (c) shows an example where the model needs to reason on both text and images together. V-CoT is able to fix this example.



Are there more big red rubber double buss in front of the large red double bus than big green things?
Choices:
(A) Yes
(B) No

In the image, there is one large red double bus and no big red rubber double buses in front of it. There is one big green car. Since there are no red buses in front of the red double bus to compare with the green car, we can conclude that there are not more big red rubber double buses in front of the large red bus than big green things. Answer: (B) No

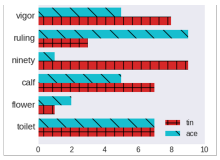
(a) Correct Answer w/ Correct Reasoning



As shown in figure, AB and CD intersect at point O, OD bisects ∠AOE, ∠BOC = 50.0, then ∠EOB =
Choices:
(A) 50° (B) 60°
(C) 70° (D) 80°

The image displays the intersection of straight lines AB and CD at point O, with OD bisecting ∠AOE and an angle ∠BOC given as 50. In the image, ∠BOC = 50, then ∠AOD also equals 50. The angles in a straight line sum up to 180, so to find ∠EOB, we subtract 50 from 180 (since ∠EOB is supplementary to ∠BOC), giving us:∠EOB = 180 - 50 = 130. However, the answer choices do not include 130.

(b) Incorrect Answer w/ Incorrect Reasoning



What is the sum of all values in the ruling group?

Choices:
(A) 10 (B) 12
(C) 14 (D) 8

The image shows a bar graph with different categories, the bars do not have precise scale markings for each numbered division. I will estimate the values by sight. For the "ruling" category, there appear to be about 6 tin (red) blocks and about 4 ace (blue) blocks. The total sum of values for the "ruling" category appears to be 6 (for tin) + 4 (for ace) = 10. Answer: (A) 10

(c) Incorrect Answer w/ Correct Reasoning

Figure 7: Observed patterns in GPT-4V + V-CoT responses to MathVista tasks. (a) shows an example where both the reasoning and answer are correct; (b) shows an example where both the reasoning and answer are incorrect as the model misses to subtract angle DOE; (c) shows an example where the answer is incorrect but the reasoning is correct, as the model correctly figures out the correct bar but cannot generate the correct value from scale.

B.4 Code Generation

On SQL generation for Spider tasks, we make the following observations:

- Structural understanding. The rendered table provides improvements over the text representation of the same table for tasks that require reasoning over the structure of a table (pivot, group, transposing). An example is shown in Figure 9a.
- Token efficiency of images. Considering tables as images consumed fewer tokens than their corresponding text description produced by InstructBlip.
- Reasoning over cell values. Tasks that require reasoning over cell values, such as filtering or splitting, benefit from a textual representation of tables or from the image paired with v-CoT (see Figure 9b).

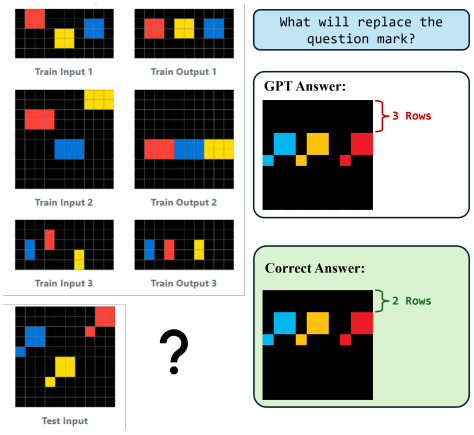


Figure 8: In ARC, GPT-4V can still struggle in identifying/placing blocks in appropriate locations (like offsets). The figure shows that GPT-4V correctly identifies that it has to align the blocks to the level of the blue block, but it is offset by one block.

Location	Name	Country	Age
Raith Rovers	Joe Sharp	Netherlands	52
Ayr United	Timbaland	United States	32
East Fife	Justin Brown	France	29
Queen's Park	Rose White	France	41
Arbroath	Tribal King	France	25

Question:
Show all countries and the number of singers in each country.

GPT Answer:
```
SELECT country , count(*)
FROM singer GROUP BY country
```

Correct Answer:
```
SELECT country , count(*)
FROM singer GROUP BY country
```

(a) Success Case

Dept. ID	Name	Creation	Ranking	Budget
1	State	1789	1	9.96
2	Treasury	1789	2	11.1
3	Defense	1947	3	439.3
4	Justice	1870	4	23.4
5	Interior	1849	5	10.7

Question:
What is the budget for agriculture

GPT Answer:
```
SELECT Budget FROM department
where Name = 'agriculture'
```

Correct Answer:
```
SELECT Budget FROM department
where Name='Agriculture/Land'
```

(b) Failure Case

Figure 9: We show success and failure cases of adding tables as images. (a) Adding the table image can help for queries that requiring overall table structure (like grouping) or extracting particular values (like filtering) from table rendering. (b) For queries that need constant values, GPT-4V struggles to get it from the image.

C Prompting Results

D Data Generation

D.1 Code Generation

We use (Zhong et al., 2017) as the original source dataset. The source dataset contains (`text instruction, table, SQL query`) triplets. The tables are orginally in `HTML` encoding. To generate images, we use `matplotlib` to convert the tables to image.

To generate structured propositions, we extract special constants from a subset of SQL statements and create structured propositions with 2 properties *Has Name* and *Has Value*. Specifically,

Approach	MathVista	ChartQA	ARC	Spider
Sphinx	33.2	63.4	27.2	-
InstructBlip	24.7	54.5	23.4	-
Blip2	23.1	31.4	22.6	-
InstructBlip + CoT GPT-4	30.4	55.3	47.3	83.5
InstructBlip + PoT GPT-4	30.8	56.4	**51.2**	82.7
GPT-4	-	-	-	81.8
GPT-4V	47.5	75.6	31.2	84.3
GPT-4V + v-CoT	**49.1**	**79.2**	40.5	**86.2**

Table 5: Results over 20% of tasks sampled from each benchmark uniformly at random. InstructBlip + GPT-4 indicates captioning by InstructBlip and reasoning over this by text GPT-4. CoT represents Chain of Thought; PoT represents Program of Thought. GPT-4 does not use a vision component and is only evaluated on problems that do not depend on an image.

- column names from `SELECT` statements to produce *Has Name* propositions
- constant values in conditional operations (e.g. `WHERE` statements) to produce *Has Value* propositions.

Example 7. Consider the SQL query `SELECT "Name" from "Students" where "ID" == "X12"`, our proposition mining will return (Table Students, Has Name, "Name"), (Name, Has Value, X12), (Table Students, Has Name, ID).